# LexAI: Complete Local Setup & Execution Guide for Team Members

**Platform:** Windows 10 / 11 | **IDE:** Visual Studio Code | **Stack:** Python 3.10–3.12, Flask, MySQL, ChromaDB, Google Gemini API

---

## 1. Prerequisites & Required Software

Before setting up LexAI, ensure your Windows laptop has the required runtime software installed.

### 1.1 Python (Version 3.10 to 3.12, 64-bit)

- **Why:** LexAI utilizes PyTorch, SentenceTransformers ( `all-MiniLM-L6-v2` ), and ChromaDB. Python 3.10, 3.11, or 3.12 provide pre-compiled wheels for Windows without needing Visual Studio C++ build tools.
- **Download:** python.org/downloads
- > [!IMPORTANT] > During Python installation on Windows, **YOU MUST CHECK** the box:
  > > ☑ `Add python.exe to PATH` (at the bottom of the installer window).
- **Verification:** Open Windows PowerShell and run: `powershell python --version` *(Expected output:* `Python 3.10.x` , `Python 3.11.x` , *or* `Python 3.12.x` *)*

---

### 1.2 Visual Studio Code (VS Code)

- **Download:** code.visualstudio.com
- **Recommended Extensions:**
  - **Python** (by Microsoft)
  - **Pylance** (by Microsoft)
- **Verification:** `powershell code --version`

---

### 1.3 MySQL Server & MySQL Workbench

- **Download:** MySQL Community Installer (Windows) (Choose the `mysql-installer-community` ~300 MB package).
- **Components to Select during install:**
  1. **MySQL Server 8.0, 8.4, or 9.x**
  2. **MySQL Workbench** (graphical UI for databases)
- **Setup Password:** Note down the password you set for the default `root` user during installation (e.g. `root123` or your personal password).
- **Verification:** Verify MySQL service is running on Windows: `powershell Get-Service -Name "*mysql*"` *(Status should say* `Running` *, e.g.* `MySQL80` , `MySQL84` , *or* `MySQL96` *)*

---

### 1.4 Git (Optional)

- Useful if you want to track changes or clone directly.
- **Download:** git-scm.com/download/win
- **Verification:** `git --version`

---

## 2. Extract and Open the Project in VS Code

### 2.1 Extracting the Project

1. Locate the `LexAI.zip` file on your computer (e.g. in your `Downloads` folder).
2. Right-click `LexAI.zip` and select **Extract All...**.
3. Choose an easy path, for example: `C:\Projects\LexAI` or `C:\Users\<YourUsername>\Documents\LexAI`. *(Avoid deep paths with unusual characters or OneDrive sync folders).*

### 2.2 Opening in VS Code

1. Open **VS Code**.
2. Go to **File > Open Folder...** (or press `Ctrl + K, Ctrl + O`).
3. Select the extracted project folder where `app.py` is located.
4. Open the integrated PowerShell terminal in VS Code:
   - Press `Ctrl + ` ` (Backtick) or click **Terminal > New Terminal**.

### 2.3 Verify Directory Structure

In the integrated terminal, check your current location:

```
Get-Location
```

Verify that the core project files exist:

```
Get-ChildItem -Name
```

You should see: * `app.py` (Application entrypoint) * `config.py` (Configuration loader) * `requirements.txt` (Python package list) * `models/` (SQLAlchemy database models) * `routes/` (Flask Blueprints: auth, doc, chat, admin, main) * `services/` (Document processing, RAG, AI, embeddings) * `templates/` & `static/` (Jinja2 HTML templates and frontend assets)

---

## 3. Create and Activate a Virtual Environment

A Python Virtual Environment keeps LexAI's dependencies isolated from other Python projects.

### 3.1 Create Virtual Environment

Run this in the VS Code terminal (project root):

```
python -m venv venv
```

*(Wait 10–20 seconds for the `venv` folder to be generated).*

### 3.2 Handling PowerShell Script Execution Policy

Windows PowerShell may block running scripts by default with the error:
`cannot be loaded because running scripts is disabled on this system`.

To allow script execution safely for your current terminal session only:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

*(This is safe and only applies to the current active terminal session).*

### 3.3 Activate the Virtual Environment

```
.\venv\Scripts\Activate.ps1
```

When activated, your terminal prompt will be prefixed with `(venv)`:

```
(venv) PS C:\Projects\LexAI>
```

### 3.4 Verify the Active Python Interpreter

Ensure your commands are using the virtual environment's Python, not the global system Python:

```
Get-Command python | Select-Object -ExpandProperty Source
```

*(The path should end in `\LexAI\venv\Scripts\python.exe`).*

> [!TIP] In VS Code, press `Ctrl + Shift + P`, type **Python: Select Interpreter**, and select `Python ('venv': venv) .\venv\Scripts\python.exe`. This ensures VS Code autocomplete and linting link directly to your virtual environment.

---

## 4. Install Python Dependencies

### 4.1 Upgrade Pip

```
python -m pip install --upgrade pip
```

### 4.2 Install Packages from `requirements.txt`

```
pip install -r requirements.txt
```

**What gets installed:**

- **Flask 3.0.0, Flask-SQLAlchemy, Flask-Login, Flask-Bcrypt**: Core web framework, ORM, authentication, and secure password hashing.
- **PyMySQL**: MySQL database driver.
- **PyMuPDF & python-docx**: Document text extraction from PDF and DOCX files.
- **google-genai**: Official Google Gemini SDK for legal document analysis and answer generation.
- **chromadb**: Local vector database for semantic search and RAG retrieval.
- **sentence-transformers**: Local embeddings generation (`all-MiniLM-L6-v2`).

- **pytest**: Automated testing suite.

**Troubleshooting Package Installation:**

- **Network Timeout:** If downloading `sentence-transformers` or PyTorch times out, run: `powershell pip install --default-timeout=1000 -r requirements.txt`
- **Long Path Error on Windows:** If pip complains about filename length, open PowerShell as Administrator and run: `powershell New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force`

---

## 5. Configure MySQL Server

### 5.1 Check MySQL Service Status

Make sure MySQL is running in Windows:

```
Get-Service -Name "*mysql*"
```

If the status is `Stopped`, start it in an Administrator PowerShell:

```
Start-Service -Name MySQL80   # Replace with your actual service name, e.g. MySQL96 or MySQL
```

*(Alternatively, press `Win + R`, type `services.msc`, locate MySQL, and click "Start").*

### 5.2 Create the Required Databases

LexAI requires two databases: 1. `lexai_db` — The primary production/application database. 2. `lexai_test` — The isolated test database used when running `pytest`.

You can create them using **MySQL Command Line Client**, **MySQL Workbench**, or PowerShell.

#### Option A: Using the MySQL CLI

Open PowerShell or the "MySQL 8.0 Command Line Client":

```
mysql -u root -p
```

Enter your MySQL password, then execute:

```
CREATE DATABASE IF NOT EXISTS lexai_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS lexai_test CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
SHOW DATABASES;
EXIT;
```

#### Option B: Using the Automated Setup Script

LexAI includes a script (`setup_db.py`) that connects to MySQL and creates these databases automatically. Once you configure your `.env` file (explained in Section 6), you can simply run:

```
python setup_db.py
```

---

## 6. Configure the `.env` File (Environment Variables)

The `.env` file contains your private local credentials. It is never committed to Git and is not transferred inside ZIP archives.

### 6.1 Creating the File

1. In VS Code, right-click the root folder in the file explorer and click **New File**.
2. Name the file exactly `.env` (make sure it does not end with `.txt` or `.env.example`).
3. Copy and paste the template below into `.env`.

### 6.2 `.env` Template

```
# Flask Configuration
FLASK_DEBUG=True
SECRET_KEY=change_this_to_a_random_secret_key_123456

# MySQL Database Configuration
DB_USER=root
DB_PASSWORD=YOUR_ACTUAL_MYSQL_PASSWORD
DB_HOST=localhost
DB_PORT=3306
DB_NAME=lexai_db

# Google Gemini AI Configuration
AI_PROVIDER=gemini
GEMINI_API_KEY=YOUR_GEMINI_API_KEY_HERE
AI_MODEL=gemini-3.6-flash
```

### 6.3 Field-by-Field Explanation

| Variable | Description | Where to get it / What to enter |
|---|---|---|
| `FLASK_DEBUG` | Enables auto-reloading and debug traceback on error. | Set to `True` for development. |
| `SECRET_KEY` | Encrypts session cookies for Flask-Login. | Any long random string. Generate one via: `python -c "import secrets; print(secrets.token_hex(24))"` |
| `DB_USER` | Your local MySQL username. | Usually `root`. |
| `DB_PASSWORD` | **YOUR** local MySQL password. | The password you configured when installing MySQL on your laptop. |
| `DB_HOST` | Host address of MySQL. | `localhost` |
| `DB_PORT` | Port of MySQL. | `3306` (default MySQL port). |
| `DB_NAME` | Database name expected by LexAI. | **Must be** `lexai_db`. |
| `AI_PROVIDER` | Backend AI provider. | `gemini` |
| `GEMINI_API_KEY` | Your Google Gemini API Key. | Get a free key at aistudio.google.com. |
| `AI_MODEL` | Gemini model name. | `gemini-3.6-flash` (or `gemini-1.5-flash` / `gemini-2.0-flash`). |

> [!NOTE] **Special Characters in MySQL Passwords:**
> If your MySQL password contains characters like `@`, `#`, or `%`, LexAI automatically handles them safely using `urllib.parse.quote_plus` in `config.py`. You do not need to manually URL-encode it in `.env`.

---

## 7. Protect Environment Variables & Secrets

LexAI already has a `.gitignore` file that includes:

```
.env
venv/
chroma_db/
chroma_db_test/
uploads/
__pycache__/
```

- **Verification:** Run `git status`. Ensure `.env` is **NOT** listed under untracked files.
- **Rule:** Never send your `.env` file over email, GitHub, or public channels. Each team member uses their own `.env` file with their local credentials.

---

## 8. Initialize Database Tables & Admin Account

Once `.env` is saved and MySQL is running:

### Step 8.1: Create Databases (if not done manually)

```
python setup_db.py
```

*(Expected output: `Databases created successfully.`)*

### Step 8.2: Create Database Tables

LexAI uses SQLAlchemy models to generate all 6 tables (`users`, `documents`, `document_analysis`, `conversations`, `messages`, `knowledge_base`). Run the initialization script:

```
python init_db.py
```

*(Expected output: `Database tables initialized. Current Tables in DB: ['conversations', 'document_analysis', 'documents', 'knowledge_base', 'messages', 'users']`)*

### Step 8.3: Verify MySQL Connection

Run this one-line check to verify Flask connects to MySQL:

```
python -c "from app import create_app; from extensions import db; from sqlalchemy import text
```

### Step 8.4: Create an Admin Account

To access the Admin Panel (`/admin`) and manage the Legal Knowledge Base, create an initial admin:

```
python create_admin.py
```

Enter your name, email (e.g. `admin@lexai.local`), and an 8+ character password.

---

## 9. Transfer Existing Database Data (Knowledge Base & Test Data)

When copying a project ZIP file, **MySQL database records are not transferred automatically** because MySQL stores data in its own system folder (`C:\ProgramData\MySQL`).

### 9.1 What Data Needs to Be Transferred?

- **Knowledge Base Articles (`knowledge_base` table):** The project contains curated Indian legal knowledge base articles used for RAG responses.
- *Note:* Private user accounts and uploaded documents should **not** be transferred for privacy.

### 9.2 How the Team Leader Exports the Knowledge Base

On the team leader's laptop:

```
mysqldump -u root -p lexai_db knowledge_base > lexai_knowledge_base.sql
```

*(This produces a small, shareable SQL file containing only the legal articles).*

### 9.3 How the Team Member Imports the SQL File

On the team member's laptop: 1. Copy `lexai_knowledge_base.sql` into the project root. 2. Import it using PowerShell: `powershell Get-Content lexai_knowledge_base.sql | mysql -u root -p lexai_db` *(Or open `lexai_knowledge_base.sql` in MySQL Workbench and click Execute).* 3. Verify the imported articles: `powershell python -c "from app import create_app; from models.database_models import KnowledgeBase; app = create_app(); with app.app_context(): print('Total KB Articles in DB:', KnowledgeBase.query.count())"`

---

## 10. Configure ChromaDB & The RAG Pipeline

LexAI uses **ChromaDB** as its vector database and `SentenceTransformer('all-MiniLM-L6-v2')` for generating 384-dimensional mathematical embeddings.

### 10.1 Where ChromaDB Stores Vectors

- ChromaDB stores its data locally in the `chroma_db/` directory in the project root ( `chroma.sqlite3` ).

### 10.2 First-Time Embedding Model Download

When you run LexAI or indexing for the first time, `sentence-transformers` automatically downloads the lightweight `all-MiniLM-L6-v2` model (~80 MB) from Hugging Face into your user cache directory. Once downloaded, it runs 100% locally and offline without internet or API keys.

### 10.3 Rebuilding Vector Indexing from MySQL (Zero API Cost)

If your `chroma_db/` folder is empty or was not included in the ZIP, you can index all existing Knowledge Base articles from MySQL into ChromaDB **without spending any Gemini API quota**:

Run this one-line re-indexing command:

```
python -c "from app import create_app; from models.database_models import KnowledgeBase; from
```

> [!WARNING] **DO NOT** run `python scripts/seed_kb.py` unless you want to re-generate articles from scratch. `seed_kb.py` sends 93 separate prompt requests to Gemini, which will quickly exhaust free tier API rate limits. Use the SQL import + local re-index command above instead!

### 10.4 Test Local RAG Retrieval

To verify that ChromaDB and the RAG service can retrieve and formulate legal answers:

```
python test_rag_local.py
```

*(If you see `SUCCESS: {'answer': '...', 'sources': [...]}`, your RAG pipeline is working!).*

---

## 11. Run the Application

### 11.1 Start the Server

Make sure your virtual environment is active `(venv)` and run:

```
python app.py
```

### 11.2 Expected Terminal Output:

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://<Your-IP>:5000
Press CTRL+C to quit
```

### 11.3 Open the Web Application

Open your browser (Chrome, Edge, or Firefox) and navigate to:

🌐 **http://127.0.0.1:5000**

### 11.4 Stopping the Server

To stop the Flask server, click inside the terminal and press: `Ctrl + C`

---

## 12. Feature Verification Checklist

Perform these quick tests to confirm everything works:

| Feature | Action | Expected Successful Result |
|---|---|---|
| **Homepage** | Open `http://127.0.0.1:5000` | Clean landing page with "Login" and "Get Started" buttons. |
| **Registration** | Click "Sign Up", fill form, submit | Redirects to Login page with success alert. |
| **Login** | Log in with registered user | Redirects to Dashboard with user name in navigation bar. |
| **Document Upload** | Upload sample `.pdf` or `.docx` contract | Progress bar completes; file appears in document list with `completed` status and word/page counts. |
| **AI Analysis** | Click "Analyze Document" on uploaded file | Gemini analyzes contract and displays structured executive summary, risk analysis, and obligations. |
| **Document Chat** | Open chat for the uploaded document; ask a question | AI answers strictly based on the clauses in the uploaded contract. |
| **General Legal Chat** | Open "General Legal Assistant"; ask: "What is an NDA?" | AI answers using retrieved context from Knowledge Base articles. |
| **Admin Panel** | Log in with admin account, navigate to `/admin` | Admin dashboard loads showing system stats and KB management tools. |
| **Automated Tests** | In a separate terminal, run: `pytest tests/` | All test suites pass against the isolated `lexai_test` database. |

---

## 13. Practical Troubleshooting Guide

| Issue / Error Message | Likely Cause | Solution / Fix |
|---|---|---|
| `python : The term 'python' is not recognized` | Python not added to Windows PATH. | Re-run Python installer, choose "Modify", and check `Add python.exe to PATH`. Or specify full path `C:\Users\<User>\AppData\Local\Programs\Python\Python312\python.exe`. |
| `Activate.ps1 cannot be loaded because running scripts is disabled` | PowerShell execution policy restriction. | Run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process` in the current terminal, then retry `.\venv\Scripts\Activate.ps1`. |
| `Can't connect to MySQL server on 'localhost' (10061)` | MySQL Windows Service is not running. | Run `Start-Service -Name MySQL80` (or `MySQL96`) in Administrator PowerShell, or start it via `services.msc`. |
| `Access denied for user 'root'@'localhost' (using password: YES)` | Incorrect MySQL password in `.env`. | Open `.env` and verify `DB_PASSWORD` matches your local MySQL root password. |
| `Unknown database 'lexai_db'` | Database has not been created yet. | Run `python setup_db.py` or execute `CREATE DATABASE lexai_db;` in MySQL. |
| `ModuleNotFoundError: No module named 'flask'` | Virtual environment is not activated. | Run `.\venv\Scripts\Activate.ps1`. Verify `(venv)` appears in the prompt. |
| `GEMINI_API_KEY is not configured` | Missing or empty key in `.env`. | Add `GEMINI_API_KEY=your_key` in `.env` and restart `python app.py`. |
| `429 Resource Exhausted / Quota Exceeded` | Gemini API free tier rate limit reached. | LexAI automatically uses local Knowledge Base fallback! Wait 60 seconds or switch `AI_MODEL` in `.env`. |
| `Address already in use (Port 5000)` | Another process is using port 5000. | Find and stop the old process: `Get-Process python \| Stop-Process -Force` or change port in `app.py` line 64: `port=5001`. |
| `ChromaDB / sqlite3 error` | Corrupt or mismatched vector database. | Delete the `chroma_db/` folder and re-run the zero-cost indexing command from Section 10.3. |

## 14. Daily Quick-Start Checklist

After completing the initial setup above, your regular daily startup workflow is simple:

1. Open VS Code in the LexAI project folder.
2. Open PowerShell terminal ( `Ctrl + ` ).
3. Activate virtual environment: `powershell .\venv\Scripts\Activate.ps1`
4. Start LexAI: `powershell python app.py`
5. Open your browser at **http://127.0.0.1:5000**.
6. When finished, press `Ctrl + C` in the terminal to stop the server.